Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Dizilerin (array) kısa tekrarı: indisleme türleri, statik ve dinamik diziler, ekleme/silme maliyeti.
- Bağlantılı liste (linked list) kavramı: düğüm (node), veri ve `next` işaretçisi, `head` işaretçisi.
- C'de düğüm tanımı (`typedef struct ...`), listeyi `for` döngüsüyle gezme.
- Listeye eleman ekleme: başa, araya, sona; sıralı listeye ekleme.
- İki dizi (`A` veri dizisi, `L` link dizisi) ile linked list gösterimi ve akış diyagramları.
- Listeden eleman silme ve `free` ile belleği geri verme.
- Linked list ile insertion sort.
- Çift yönlü bağlantılı liste (doubly linked list): ekleme ve silmede işaretçi güncelleme sırası.
- Dairesel bağlantılı liste (circular linked list) ve Josephus (Romalı) problemi.
- Dizi ile linked list karşılaştırması.

Hocanın Özellikle Vurguladığı Kısımlar

- C'de dizi sınırlarını ve indisleri kontrol etmek programcının sorumluluğudur. Dikkatsiz kod başka bellek alanlarını okuyup yazabilir.
- Sıralı diziye araya eleman eklemek ya da aradan silmek kaydırma gerektirir. En kötü durumda $O(n)$ işlem yapılır.
- Linked list'te yer bulunduktan sonra ekleme ve silme sabit sayıda işlemle, yani $O(1)$ ile yapılır.
- Listeye erişmek için yalnız ilk düğümün adresini (`head`) bilmek yeterlidir. Son düğümün `next` alanı `NULL` gösterir.
- Araya eklemede işaretçi güncelleme sırası önemlidir. Önce yeni düğümün `next`'i bağlanır, sonra öncekinin `next`'i yeni düğüme çevrilir. Sıra ters olursa listenin geri kalanı kaybedilir.
- Silinen düğüm mutlaka `free` edilmelidir. Aksi hâlde program bitene kadar bellekte erişilemeyen bir alan kalır.
- Linked list'in zayıf noktası aramadır. Ortadaki elemana doğrudan gidilemediği için arama doğrusaldır ($O(n)$).
- Doubly linked list'te bir bağlantıyı ikiye bölmek için dört işaretçi güncellemesi gerekir. Bunlar belirli bir sırayla yapılmalıdır.
- Uygun veri yapısı algoritmayı sadeleştirir. Josephus problemi circular linked list ile tek döngüyle, hiç `if` kullanmadan çözülür.
- Hoca, linked list fonksiyonlarını bir kütüphane olarak yazmayı önerdi. Ödevlerde tekrar tekrar kullanılabilir.

Kısa Tekrar Notları

- **Sıfır tabanlı indisleme:** C'de ilk eleman `a[0]`, son eleman `a[n-1]`. Pascal gibi dillerde bir tabanlı indisleme var. Python negatif indisle sondan erişime izin verir.
- **Akış diyagramları:** Genellikle `1..n` ile yazılır. C koduna çevirirken döngü `0..n-1` olur.
- **Statik dizi:** Boyut derleme öncesinde belli olur, örneğin `int a[10];`.
- **Dinamik dizi:** `malloc` ile alınır, boyut `realloc` ile değiştirilir.
- **Karakter dizisi:** `"dizi"` yazılırsa sona otomatik `'\0'` eklenir ve 5 bayt yer ayrılır.
- **Düğüm:** veri + bir sonraki düğümün adresi. Liste dinamik olarak büyür ve küçülür.

- **Başa ekleme:** `tmp->next = head; head = tmp;` — $O(1)$.
- **Araya ekleme:** `tmp->next = cur->next; cur->next = tmp;` — yer bulunduktan sonra $O(1)$.
- **Aradan silme:** `b = a->next; a->next = b->next; free(b);`
- **Eleman sayısı:** Dışarıda bir sayaç tutulursa $O(1)$ ile bulunur. Her seferinde baştan sayılırsa $O(n)$ sürer.
- **Doubly linked list:** Her düğümde `prev` ve `next` vardır. Baş için `head`, son için `tail` tutulur.
- **Circular linked list:** Son düğüm `NULL` yerine `head`'i gösterir. Sonu bulmak için `NULL` değil, `head`'e dönüş kontrol edilir.
- **Dizi mi linked list mi:**
  - Ekleme ve silme hep sondaysa dizi uygundur.
  - Başa ya da araya çok ekleme/silme varsa linked list uygundur.
  - İndisle rastgele erişim gerekiyorsa dizi uygundur.
  - Sıralı veride ikili arama ($O(\log n)$) gerekiyorsa dizi uygundur.
  - Sadece sırayla erişim gerekiyorsa linked list uygundur.
  - Bellek: dizi yaklaşık n, linked list yaklaşık $2n$ birim yer kaplar (veri + işaretçi). İkisi de $O(n)$'dir.

Detaylı Açıklamalar

Diziler ve Kaydırma Maliyeti

Dizinin en önemli özelliği, her elemana indisle sabit zamanda ($O(1)$) erişilmesidir. Vektör, matris, string, stack, queue, heap gibi pek çok yapı dizinin üzerine kurulur.

C'de bir göze değer atamadan önce o gözü okumak tanımsız sonuç verir. Gözde ya sıfır ya da daha önceki kullanımdan kalmış bir değer bulunur.

Sıralı `2, 3, 6, 8, 10, ..., 24` dizisine 9 eklemek için:

1. Dizi `realloc` ile bir göz büyütülür.
2. 9'dan büyük bütün elemanlar birer göz sağa kaydırılır.
3. 9 boşalan göze yazılır.

En kötü durum başa eklemedir; bütün elemanlar kaydırılır. Maliyet $O(n)$'dir. Aradan silmede de elemanlar sola kaydırılır ve maliyet yine $O(n)$ olur.

Linked List Yapısı ve C Tanımı

Her düğüm bir veri alanı ve bir sonraki düğümü gösteren işaretçi taşır. Düğümler bellekte dağınık durabilir. Mantıksal sıra işaretçilerle kurulur.

```c
typedef struct link {
    int data;
    struct link *next;
} Node;
```

Derste önce `typedef struct node *link;` biçiminde bir işaretçi tipi tanımlandı. Ardından yapının tamamına `Node` adı verildi. İkisi de aynı işi görür.

Listeyi gezme:

```c
for (t = head; t != NULL; t = t->next)
    use(t->data);   /* örneğin printf */
```

Örnek liste 3 → 5 → 7 → 8 → `NULL` için döngü sırasıyla 3, 5, 7, 8 değerlerini kullanır. `t` `NULL` olunca döngü biter.

Eleman Ekleme

Başa ekleme (insert top):

```c
Node *insert_top(int num, Node *head) {
    Node *new_node = (Node *)malloc(sizeof(Node));
    new_node->data = num;
```

```
    new_node->next = head;
    head = new_node;
    return head;
}
```

Belirli bir değerin önüne ekleme (insert before): `cur->next->data` aranan değere eşit olana kadar ilerlenir. Önüne ekleneceği için hep bir sonrakinin verisine bakılır. İlk eleman için bu kontrol çalışmaz; o durum `insert_top` ile ayrıca ele alınır.

```
Node *insert_before(int next_num, int num, Node *head) {
    Node *cur = head;
    while (cur->next->data != next_num)
        cur = cur->next;
    Node *new_node = (Node *)malloc(sizeof(Node));
    new_node->data = num;
    new_node->next = cur->next;   /* 1. adım */
    cur->next = new_node;         /* 2. adım */
    return head;
}
```

Sona ekleme: `next`'i `NULL` olan düğüme kadar gidilir. Yeni düğümün `next`'i `NULL` yapılır, son düğümün `next`'i yeni düğüme bağlanır.

İki Dizi ile Linked List ve Sıralı Listeye Ekleme

Akış diyagramında işaretçi çizmek zor olduğu için liste iki diziyle gösterildi:

- `A`: değerler,
- `L`: her elemanın bir sonrakinin indisi (`-1` = liste sonu),
- `head`: ilk elemanın indisi.

Sıralı listeye ekleme akışı:

1. Yeni değer dizinin sonuna yazılır (`A[n] ← ekle`, `n ← n+1`).
2. `A[head] < ekle` değilse yeni eleman başa girer: `L[n-1] ← head`, `head ← n-1`.
3. Değilse `prev ← head`, `next ← L[head]` ile başlanır. `next ≠ -1` ve `A[next] < ekle` olduğu sürece `prev ← next`, `next ← L[next]` ile ilerlenir.
4. Döngüden çıkınca `L[prev] ← n-1`, `L[n-1] ← next` yapılır.

Derste örnek olarak `A = [7, 5, 3, 8]`, `L = [4, 1, 2, -1]`, `head = 3` alındı (1 tabanlı). Liste 3 → 5 → 7 → 8 sırasındadır.

Adım	İşlem	Sonuç
6 ekle	5. göze 6 yazıldı. 3 < 6 ve 5 < 6 olduğu için ilerlendi, 7 > 6'da durdu. `L[2] ← 5`, `L[5] ← 1`.	3 → 5 → 6 → 7 → 8
1 ekle	3 < 1 sağlanmadığı için başa eklendi. `L[6] ← 3`, `head ← 6`.	1 → 3 → 5 → 6 → 7 → 8

İkinci örnekte 0 tabanlı `A = [2, 4, 6, 8]` dizisine önce 1 başa, sonra 5 araya eklendi. Sonuç 1 → 2 → 4 → 5 → 6 → 8 oldu.

Eleman Silme

En basit silme `cur->next = cur->next->next;` satırıdır. Ancak silinen düğüm bellekte kalır. Doğrusu üç adımdır:

```
b = a->next;        /* silinecek düğüm */
a->next = b->next;  /* bağlantıyı atla */
free(b);            /* belleği geri ver */
```

İki diziyle sıralı listeden silme:

- Silinecek eleman `head` ise yalnız `head ← L[head]` yapılır.
- Değilse `prev` ve `adres` ile ilerlenir. `adres ≠ -1` ve `A[adres] < sil` olduğu sürece devam edilir.
- Sona gelindiyse ya da `A[adres] > sil` ise eleman listede yoktur, hata verilir.
- Bulunduysa `L[prev] ← L[adres]` yapılır.

Örnek: `2, 3, 4, 6, 7, 8` listesinden önce 2 (baş) silindi, `head` 1 oldu. Sonra 6 silindi, 4'ün linki 7'yi gösterdi. Kalan liste 3 → 4 → 7 → 8 oldu.

C'de silme fonksiyonu `current` ve `previous` işaretçileriyle yazıldı:

```
Node *delete_value(int num, Node *head) {
    Node *cur = head, *prev = NULL;
    while (cur != NULL) {
        if (cur->data == num) {
            if (cur == head)
                head = cur->next;
            else
                prev->next = cur->next;
            free(cur);   /* slaytta eksikti, hoca eklenmesini söyledi */
            return head;
        }
        prev = cur;
        cur = cur->next;
    }
    return head;
}
```

Not: Slayttaki kodda `free` yoktu. Hoca bu satırın eklenmesi gerektiğini söyledi. Yukarıdaki kodda `free` sonrası işaretçiye dokunulmasın diye fonksiyondan hemen dönüldü.

Linked List ile Insertion Sort

Girdi listesi `A`, çıktı listesi `B` olsun. Her adımda:

1. `A`'nın başındaki düğüm koparılır ($O(1)$).
2. `B`'de baştan başlanarak doğru yer bulunur.
3. Düğüm iki işaretçi güncellemesiyle araya takılır.

Örnek: `A`'daki 627, 758, 113 sırayla `B`'deki (98, 101, 515, 838, ...) uygun yerlere eklendi. Dizideki gibi kaydırma yapılmaz, yalnız bağlantılar değişir.

Çift Yönlü Bağlantılı Liste (Doubly Linked List)

```
typedef struct link {
    int data;
    struct link *next;
    struct link *prev;
} Node;
```

Aynı fikirle matris (sağ ve aşağı işaretçileri) ya da üç işaretçiyle veri küpü bile kurulabilir.

`cur`'dan sonra `new` düğümünü ekleme sırası:

1. `new->next = cur->next;`
2. `cur->next->prev = new;`
3. `cur->next = new;`
4. `new->prev = cur;`

`del` düğümünü silme sırası:

1. `del->next->prev = del->prev;`
2. `del->prev->next = del->next;`
3. `free(del);`

Dairesel Bağlantılı Liste ve Josephus Problemi

Son düğüm `NULL` yerine `head`'i gösterirse liste bir çember olur. `next` ile ilerlemek sonsuza kadar döner.

Josephus probleminde n kişi çember olur ve her m'inci kişi elenir. Hoca problemi $n = 9$, $m = 5$ ile çözdü.

Tur	Elenen	Kalanlar
1	5	1 2 3 4 6 7 8 9
2	1	2 3 4 6 7 8 9
3	7	2 3 4 6 8 9
4	4	2 3 6 8 9
5	3	2 6 8 9
6	6	2 8 9
7	9	2 8
8	2	8

Son $m - 1 = 4$ kişide durulursa 2, 6, 8, 9 kalır. Sona kalan tek kişi 8'dir.

Slayttaki kod (Sedgewick):

```
int i, N = 9, M = 5;
link t = malloc(sizeof *t), x = t;

t->data = 1; t->next = t;

for (i = 2; i <= N; i++) {
    x = (x->next = malloc(sizeof *x));
    x->data = i; x->next = t;
}

while (x != x->next) {
    for (i = 1; i < M; i++) x = x->next;
    x->next = x->next->next;
    N--;
}
printf("%d\n", x->data);
```

Döngüden önce `x` son düğümü (9) gösterir. İç döngü $M - 1$ adım ilerler, sonra bir sonraki düğüm listeden çıkarılır.

Ders Sonrası Notlar

- **Josephus kodunun karmaşıklığı:** Hoca toplam maliyeti $O(n)$ olarak verdi. Oysa her eleme için $m-1$ adım ilerleniyor ve $n-1$ eleme yapılıyor. Doğrusu $O(n \cdot m)$'dir. m sabit kabul edilirse $O(n)$'e iner.
- **Josephus kodunda bellek:** Kod, listeden çıkarılan düğümleri `free` etmiyor. Gerçek uygulamada çıkarılan düğüm önce bir değişkende tutulup serbest bırakılmalıdır.
- **Elenme sırası:** Derste ikinci elenenin söylenişi karıştı. Doğru sıra yukarıdaki tablodadır: 5, 1, 7, 4, 3, 6, 9, 2.
- **Tarihsel bilgi:** Hoca olayı "milattan önce" diye anlattı. Olay MS 67 yılında, Roma'ya karşı Yahudi isyanı sırasında geçer. Rivayete göre Josephus, Vespasianus'un imparator olacağını ona kendisi söylemiştir.
- **Tek yönlü listede sondan silme:** Hoca baştan, aradan ve sondan silmenin sabit zamanda olduğunu söyledi. Bu, silinecek düğümün öncesi biliniyorsa doğrudur. Tek yönlü listede sondan bir öncekini bulmak için baştan gezmek gerekir ve bu $O(n)$ sürer.
- **Listeyi tersine çevirme:** Tersine çevirmek kolaydır ama yine bütün düğümlerin işaretçisi güncellenir. Maliyet $O(n)$'dir.
- **Insertion sort'un maliyeti:** Linked list kaydırmayı ortadan kaldırır. Ancak her eleman için çıktı listesinde doğrusal arama yapılır. En kötü durum yine $O(n^2)$'dir.

- **`insert_before` kodu:** Aranan değer listede yoksa `cur->next` `NULL` olur ve kod çöker. Döngü koşuluna `cur->next != NULL` kontrolü eklenmelidir.

Duyurular: - Bu dönem laboratuvar dersi yok. - İlk ödev bir sonraki hafta tanımlanacak. Değerlendirmeler Nisan'a ertelendiği için teslim Mart sonu ya da Nisan başı olabilir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.